

ENGINEERING REBOOT 2026

# DAY 36 FIELD MANUAL

Inheritance Fundamentals

WEEK 08 | DAY 36 | FIELD MANUAL v1.0

Phase II - Modern Software Engineering Foundations

Publication Edition - August 2026

# Mission Brief

Today introduces inheritance as a way to represent meaningful specialization between classes. The session deliberately starts with a minimal subclass so inherited behavior is visible before constructor specialization, method overriding, or `super()` are introduced on Day 37.

## PRIMARY OBJECTIVE

By the end of Day 36, you should be able to define a base class and subclasses, demonstrate what the subclasses inherit, add subclass-specific behavior, and explain why the relationship is a legitimate is-a relationship.

## 1. Learning Objectives

- Define inheritance in object-oriented programming.
- Identify the base class and derived class in a simple hierarchy.
- Create a subclass using Python inheritance syntax.
- Demonstrate inherited attributes and methods.
- Add behavior that belongs only to a subclass.
- Distinguish an is-a relationship from convenient code reuse.

## 2. Concept Primer

**Inheritance** allows a class to acquire accessible state and behavior from another class. The existing class is the **base class** or parent; the specialized class is the **derived class** or subclass.

For Day 36, keep the model shallow: **Employee** is the general concept; **SalariedEmployee** and **HourlyEmployee** are specialized kinds of Employee.

```
Employee
|-- SalariedEmployee
`-- HourlyEmployee
```

## 3. Guided Exercise 1 - Observe Inheritance

**Suggested filename:** `inheritance_basics.py`

Create an `Employee` class with a constructor that stores an employee ID and name, plus a simple `display_info()` method. Then create a `SalariedEmployee` subclass containing only `pass`.

```
class Employee:
    def __init__(self, employee_id, name):
        self.employee_id = employee_id
        self.name = name

    def display_info(self):
        print(self.employee_id, self.name)

class SalariedEmployee(Employee):
    pass
```

### Run the following observations:

- 1 Instantiate `SalariedEmployee` using the constructor defined by `Employee`.

- 2 Access the inherited `employee_id` and `name` attributes.
- 3 Call the inherited `display_info()` method.
- 4 Confirm that the subclass works even though its class body contains no implementation beyond `pass`.

### CHECKPOINT

Be able to explain: *SalariedEmployee* can use *Employee*'s constructor and *display\_info()* because those behaviors are inherited from its base class.

## 4. Guided Exercise 2 - Add Specialized Behavior

**Suggested filename:** `employee_types.py`

Add both `SalariedEmployee` and `HourlyEmployee` subclasses. Give each subclass one small behavior that is meaningful only for that employee type. Keep the inherited constructor unchanged today.

### Possible behaviors

- `SalariedEmployee`: display a message identifying the employee as salaried.
- `HourlyEmployee`: display a message identifying the employee as hourly.
- Both subclasses continue using `Employee.display_info()` without reimplementing it.

### Questions to answer while coding

- 1 Which attributes and methods came from `Employee`?
- 2 Which behavior belongs specifically to the subclass?
- 3 Would `SalariedEmployee` still be an `Employee` if the shared implementation were removed?
- 4 Are we modeling a genuine specialization, or only trying to reuse code?

## 5. Mini-Project - Employee Hierarchy

**Suggested filename:** `employee_hierarchy.py`

Implement a compact employee hierarchy that demonstrates the Day 36 concepts. This is a concept exercise, not a payroll application.

### Requirements

- Create an `Employee` base class.
- `Employee` must contain common employee state and at least one common method.
- Create `SalariedEmployee` and `HourlyEmployee` subclasses.
- Both subclasses must demonstrate inherited behavior.
- Each subclass must contain at least one small subclass-specific behavior.
- Instantiate at least one object of each subclass and demonstrate the behaviors.
- Do not override `__init__` today.
- Do not use `super()` today.
- Do not introduce payroll calculations, persistence, menus, or unrelated features.

### Expected behavioral evidence

- The program creates both employee types successfully.
- Both employee types can use common `Employee` behavior.
- Each employee type can demonstrate its own specialized behavior.
- The output makes the inheritance relationship observable.

## 6. Engineering Reflection

Capture concise answers in Notes.md or Journal.md:

- 1 What did the subclass receive from its parent without reimplementing?
- 2 Why is 'SalariedEmployee is an Employee' a stronger justification than code reuse alone?
- 3 What would be an example of a has-a relationship that should not use inheritance?
- 4 What limitation do you notice if a subclass needs additional constructor data? Keep that observation for Day 37.

## 7. Definition of Done

- Guided exercises completed and executed.
- Employee Hierarchy mini-project completed and tested.
- Inheritance and subclass-specific behavior are demonstrated.
- Code and execution evidence are captured according to current reboot standards.
- Notes.md and Journal.md are completed.
- Required Git operations are performed and captured.
- Changes are committed and pushed using the established daily workflow.

### DAY 36 SCOPE GUARDRAIL

Stop at inheritance fundamentals. Constructor overriding, `super()`, and method overriding belong to Day 37. The goal is a clear mental model of inherited behavior, not a feature-rich employee system.